DIABETES PREDICTION USING MACHINE LEARNING

ANALYSIS AND MODELING REPORT

Table of Contents

Chapter 1 Abstract **4**

Chapter 2 Introduction **5**

Background and Motivation 5

Problem Statement 5

Objectives 6

Scope 6

Chapter 3 Methodology / Process **7**

3.1 Data Source and Description 7

3.2 Tools and Libraries 7

3.3 Data Loading and Initial Overview 8

3.4 Data Quality and Preprocessing 10

3.5 Data Partitioning and Scaling 12

Chapter 4 Explanatory Data Analysis (EDA) **13**

4.1 Outcome distribution plot 13

4.2 Feature histograms 13

4.3 Relationship Between Features 14

4.4 Outliers and Data Balance 17

4.5 Interpretation of EDA 20

Chapter 5 Modeling and Evaluation **22**

5.1 Baseline Model: Logistic Regression 22

5.2 Advanced Model: Random Forest 22

5.3 Gradient Boosting model 24

Chapter 6 Model Comparison and Diagnostics **26**

6.1 Model comparison 26

6.2 Comprehensive Random Forest evaluation 26

6.3 Confusion matrix interpretation for the Random Forest model 27

6.4 ROC curve 29

6.5 Feature importance results from Random Forest 30

Chapter 7 Practical Application **32**

7.1 Practical application (Prediction function): 32

Chapter 8 Conclusion **33**

8.1 Summary of Findings 33

8.2 Model Performance and Selection 33
8.3 Limitations 33
8.4 Future Improvements 33
8.5 Final Reflections 34

Chapter 1
Abstract

This project presents a data-driven approach to predicting diabetes using classical and ensemble machine learning methods on a well-known Pima Indians Diabetes Dataset. The dataset contains information on pregnancies, glucose levels, blood pressure, skin thickness, insulin levels, body mass index, diabetes pedigree function, and age for women of Pima Indian heritage, with a binary label indicating the presence or absence of diabetes.

The objective of the project is to build and evaluate predictive models that can distinguish between diabetic and non-diabetic individuals based on these risk factors, and to understand which features contribute most strongly to the prediction.

The work begins with loading and cleaning the dataset, including identification and treatment of implausible zero values in medical measurements. Exploratory data analysis (EDA) is conducted to understand the distribution of variables, relationships between predictors and the diabetes outcome, and the presence of class imbalance. Visualizations such as histograms, boxplots, correlation heatmaps, and pair plots reveal that diabetic cases tend to exhibit higher glucose, BMI, insulin, and age, and that some predictors are moderately correlated.

The dataset is partitioned into training and test subsets using a stratified 80–20 split to preserve the proportion of diabetic and non-diabetic cases. Three models are developed and compared: a baseline logistic regression, a random forest classifier, and a gradient boosting classifier. Performance metrics include accuracy, precision, recall, F1-score, confusion matrices, and ROC-AUC, with particular emphasis on the diabetic (positive) class due to its clinical importance.

The baseline logistic regression achieves approximately 73.4% accuracy on the test set, with better performance on non-diabetic cases than diabetics. An ensemble random forest model, configured with class weighting to address imbalance, improves the test accuracy to around 77.9%, with more favorable precision–recall trade-offs for diabetic predictions. Gradient boosting provides comparable performance.

Feature importance analysis confirms that glucose is the most influential predictor, followed by BMI, age, and the diabetes pedigree function, aligning with known medical risk factors. The confusion matrix and error analysis highlight the trade-offs between false positives and false negatives in a clinical context, underscoring the particular severity of missed diabetic cases.

Overall, the project demonstrates that standard machine learning models trained on carefully pre-processed clinical data can achieve moderate predictive performance for diabetes screening. The report concludes with discussion on limitations, suggestions for improving model performance, and reflections on the learning process.

Chapter 2
Introduction

Background and Motivation

Diabetes mellitus is a chronic metabolic disorder characterized by elevated blood glucose levels, resulting from defects in insulin secretion, insulin action, or both. Over time, uncontrolled diabetes can lead to serious complications affecting the cardiovascular system, kidneys, eyes, and peripheral nerves. Early detection of individuals at risk plays a crucial role in preventing or delaying these complications through lifestyle interventions, medication, and regular monitoring. With the increasing availability of health-related data, machine learning techniques offer promising tools for risk prediction and clinical decision support. By learning patterns from historical patient data, these models can help identify individuals more likely to develop or have undiagnosed diabetes, potentially supporting clinicians and public health initiatives.

The dataset used in this project is a widely studied benchmark, often referred to as the Pima Indians Diabetes Dataset. It consists of physiological and demographic variables for female patients of Pima Indian heritage, along with a binary indicator of diabetes. Despite its simplicity, the dataset captures several well-established risk factors and is suitable for demonstrating end-to-end machine learning workflows.

Problem Statement

The primary problem addressed in this project is:

Can we build a predictive model that accurately classifies individuals as diabetic or non-diabetic based on available physiological and demographic features?

This involves:

- Handling data quality issues such as implausible zero values.
- Accounting for class imbalance.
- Selecting and training appropriate machine learning models.
- Evaluating model performance using clinically meaningful metrics.
- Interpreting results and understanding the importance of different features.

Objectives

The key objectives are:

1. To perform thorough exploratory data analysis (EDA) of the diabetes dataset, including visualization and summary statistics.
2. To preprocess the data by handling implausible values, dealing with missingness-like zeros, and preparing it for modeling.
3. To implement and compare multiple classification models, including logistic regression and tree-based ensemble methods.
4. To evaluate models using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices, with a focus on the diabetic (positive) class.
5. To analyze feature importance and relate findings to clinical understanding of diabetes risk factors.
6. To reflect on model limitations, potential improvements, and the learning outcomes from this project.

Scope

This project focuses on:

- A single benchmark dataset.
- Supervised binary classification with standard scikit-learn models.
- Classical evaluation metrics and basic hyperparameter tuning (implicitly, via typical settings).

It does not include:

- Integration with real-time clinical systems.
- Complex deep learning architectures.
- Detailed cost-sensitive optimization or formal clinical trial validation.

Chapter 3
Methodology / Process

3.1 Data Source and Description

The dataset, typically provided as **diabetes.csv**, contains the following columns:

- **Pregnancies**
- **Glucose**
- **BloodPressure**
- **SkinThickness**
- **Insulin**
- **BMI**
- **DiabetesPedigreeFunction**
- **Age**
- **Outcome** (target variable, 1 for diabetic, 0 for non-diabetic)

Each row represents an individual patient. The dataset size (around 768 observations) and the approximate class distribution (~65% non-diabetic, ~35% diabetic) are consistent with the Pima Indians diabetes dataset.

3.2 Tools and Libraries

The analysis is implemented in Python, primarily using:

- **pandas** for data manipulation and exploration.
- **numpy** for numerical computations.
- **matplotlib** and **seaborn** for visualizations.
- **scikit-learn** for model training, evaluation, and utilities such as train–test split, classification metrics, and ensemble algorithms.

These are standard tools for data science and machine learning projects.

3.3 Data Loading and Initial Overview

3.3.1 Data loading

The dataset was loaded into a pandas DataFrame using the **read_csv** function:

```
df = pd.read_csv('diabetes.csv')
```

This reads the diabetes dataset from a CSV file into memory, allowing for further cleaning, exploration, and modeling.

3.3.2 Initial data overview

To obtain a quick understanding of the dataset structure, its shape, the first few records, and the data types are inspected:

```
print("Shape:", df.shape)
```

```
Shape: (768, 9)
```

```
print(df.head())
```

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64
 8   Outcome                   768 non-null    int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
```

df.shape reports the number of rows and columns in the dataset (i.e., sample size and number of features).
df.head() shows the first five observations, giving an immediate sense of the variables and their typical values.
df.info() summarizes the column names, data types, and non-null counts, which is useful to:

- Check that all variables were correctly interpreted as numeric where expected.
- Detect any missing values or incorrectly typed fields.

Given the provided **diabetes.csv**, the dataset includes the following variables:

- Pregnancies
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age
- Outcome (target variable indicating presence or absence of diabetes)

The initial inspection confirmed that the dataset is cleanly structured, with all features in numeric format and ready for exploratory data analysis and modeling.

3.4 Data Quality and Preprocessing

3.4.1 Descriptive Statistics and Data Quality

A preliminary descriptive analysis was conducted to understand the distribution of the variables in the diabetes dataset. The dataset comprises 768 records with 9 numerical variables, including patient characteristics (e.g., age, BMI, number of pregnancies, glucose, blood pressure, insulin, skin thickness) and a binary outcome variable indicating the presence of diabetes.

```
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
print(df.describe().T)
```

```
                         count        mean         std     min       25%       50%        75%     max
Pregnancies              768.0    3.845052    3.369578   0.000   1.00000    3.0000    6.00000   17.00
Glucose                  768.0  120.894531   31.972618   0.000  99.00000  117.0000  140.25000  199.00
BloodPressure            768.0   69.105469   19.355807   0.000  62.00000   72.0000   80.00000  122.00
SkinThickness            768.0   20.536458   15.952218   0.000   0.00000   23.0000   32.00000   99.00
Insulin                  768.0   79.799479  115.244002   0.000   0.00000   30.5000  127.25000  846.00
BMI                      768.0   31.992578    7.884160   0.000  27.30000   32.0000   36.60000   67.10
DiabetesPedigreeFunction 768.0    0.471876    0.331329   0.078   0.24375    0.3725    0.62625    2.42
Age                      768.0   33.240885   11.760232  21.000  24.00000   29.0000   41.00000   81.00
Outcome                  768.0    0.348958    0.476951   0.000   0.00000    0.0000    1.00000    1.00
```

On average, participants had approximately 3.85 pregnancies, with most women having between 1 and 6 pregnancies.
The mean age was 33.2 years, with an interquartile range from 24 to 41 years, indicating a relatively young adult population but with some older individuals up to 81 years.
The mean fasting plasma glucose concentration was 120.9 mg/dL (IQR 99-140.3), and the mean BMI was 32.0 kg/m², placing the typical participant in the obese category.
The diabetes pedigree function, which captures hereditary risk, had a mean of 0.47 and ranged from 0.078 to 2.42.
The outcome variable had a mean of 0.35, indicating that around 35% of the patients are diabetic, while 65% are non-diabetic.

3.4.2 Data quality and potential missing values

```
print("\nZero values per column:")
print((df == 0).sum())
```

```
Zero values per column:
Pregnancies                 111
Glucose                       5
BloodPressure                35
SkinThickness               227
Insulin                     374
BMI                          11
DiabetesPedigreeFunction      0
Age                           0
Outcome                     500
dtype: int64
```

To assess data quality, the number of zero values in each feature are counted. For variables such as Glucose, BloodPressure, SkinThickness, Insulin, and BMI, a

value of zero is not physiologically plausible and is therefore interpreted as a proxy for missing data. There were 5 zeros in Glucose, 35 in BloodPressure, 227 in SkinThickness, 374 in Insulin, and 11 in BMI. This suggests that the dataset contains a considerable amount of missing information, particularly for skin thickness and insulin measurements, which needs to be addressed during preprocessing. In contrast, zeros in Pregnancies are clinically valid (indicating women who have never been pregnant), and the Age and DiabetesPedigreeFunction variables contain no zeros at all. The 500 zeros in the Outcome variable correctly represent non-diabetic cases rather than missing data. Overall, the dataset is suitable for analysis but requires careful handling of variables with implausible zero values before modeling.

3.4.3 Data Cleaning / Preprocessing

For several physiological measurements in the dataset (Glucose, BloodPressure, SkinThickness, Insulin, and BMI), a recorded value of zero is not clinically plausible and is likely used as a placeholder for missing data. To address this, treated zeros in these variables are treated as missing values and are imputed using the median of the non-zero observations for each respective feature.
For each column, the median value of non-zero entries is computed, and all zero values are subsequently replaced with this median. Median imputation is robust to outliers and preserves the central tendency of the observed data without being overly influenced by extreme values.

```
cols_with_zero= ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in cols_with_zero:
median_val = df[col][df[col] != 0].median()
df.loc[df[col] == 0, col] = median_val

print(df.isna().sum())
```

```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

It returned zeros for all columns which ensures that the dataset is complete and suitable for downstream statistical analyses and machine learning models that cannot handle missing data directly.
Replacing implausible zeros with the median of the observed values helps maintain the overall distribution of each variable while avoiding the loss of data that would occur if rows with missing values were simply dropped. However, it is important to note that any imputation introduces some uncertainty, and this should be considered when interpreting the final model results.

3.5 Data Partitioning and Scaling

3.5.1 Train-test split and feature scaling

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Split features and target
X = df.drop('Outcome', axis=1)
y = df['Outcome']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Show shapes of resulting arrays
print(X_train.shape)
```

```
(614, 8)
```

```
print(X_test.shape)
```

```
(154, 8)
```

```
print(y_train.shape)
```

```
(614,)
```

```
print(y_test.shape)
```

```
(154,)
```

3.5.2 Data partitioning

The dataset was partitioned into predictors (X) and target variable (y), where Outcome represents the binary diabetes status and was used as the target. The remaining 8 variables were used as features.

A stratified train-test split was applied with 80% of the data assigned to the training set and 20% to the test set (test_size = 0.2, stratify = y, random_state = 42). This resulted in 614 training samples and 154 test samples, preserving the original class distribution of the outcome variable across both sets.

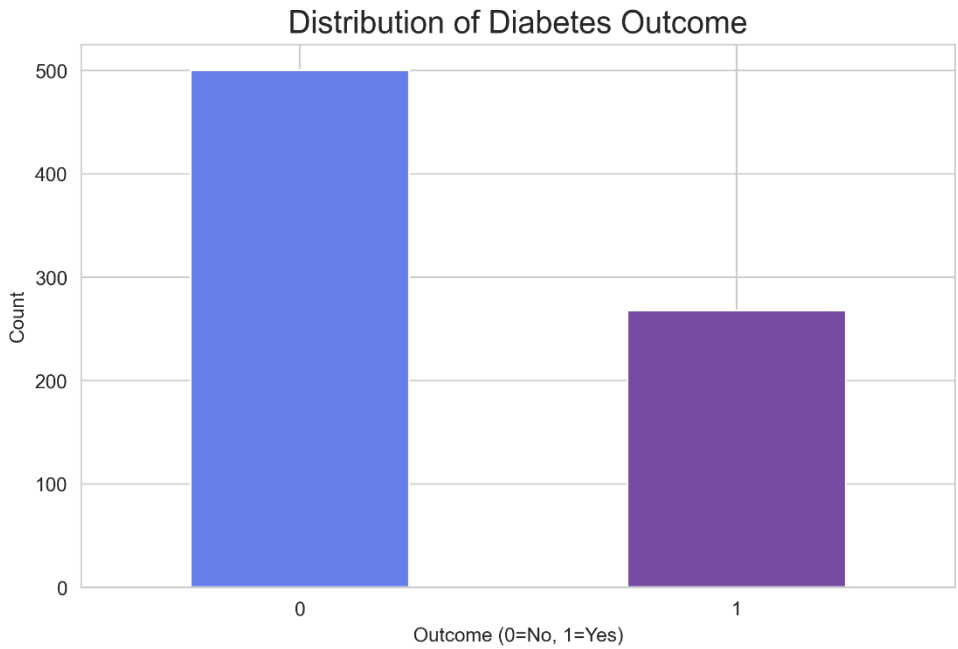
3.5.3 Feature scaling

All predictor variables were standardized using StandardScaler. The scaler was fitted on the training data (X_train) and subsequently applied to both training and test sets (X_train_scaled, X_test_scaled). Standardization ensures that features have zero mean and unit variance, which is essential for many machine learning algorithms that are sensitive to differences in feature scales.

Chapter 4
Explanatory Data Analysis (EDA)

4.1 Outcome distribution plot

```
import seaborn as sns

# Set style
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 8)

# Outcome distribution
plt.figure(figsize=(8, 5))
df['Outcome'].value_counts().plot(kind='bar', color=['#667eea', '#764ba2'])
plt.title('Distribution of Diabetes Outcome', fontsize=16)
plt.xlabel('Outcome (0=No, 1=Yes)')
plt.ylabel('Count')
plt.xticks(rotation=0)
plt.show()
```

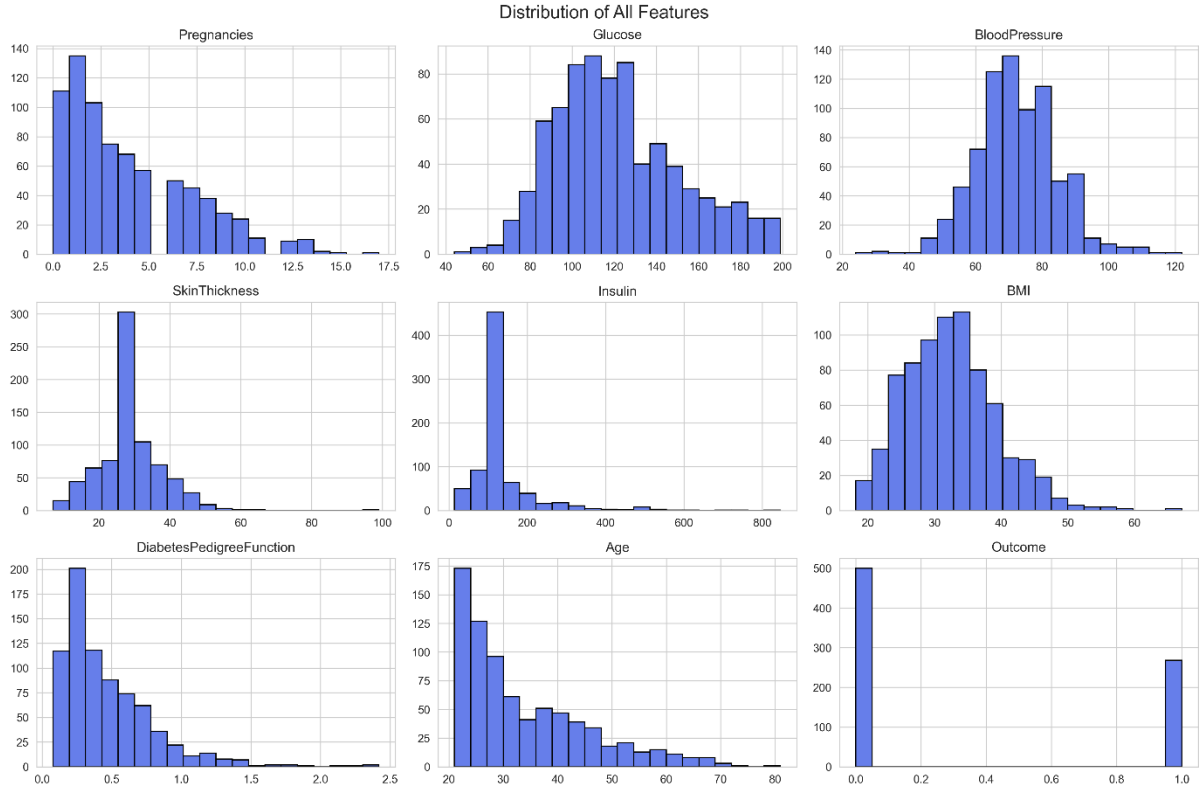


The distribution of the target variable Outcome was examined using a bar chart. Outcome=0 represents non-diabetic cases and Outcome=1 represents diabetic cases.

The plot shows a clear class imbalance, with non-diabetic cases substantially more frequent than diabetic cases. This imbalance needs to be considered during model evaluation and selection, as accuracy alone may be misleading and metrics such as precision, recall, F1-score, and ROC-AUC are more informative in this context.

4.2 Feature histograms

```
df.hist(bins=20, figsize=(15, 10), color='#667eea', edgecolor='black')
plt.suptitle('Distribution of All Features', fontsize=16)
plt.tight_layout()
plt.show()
```

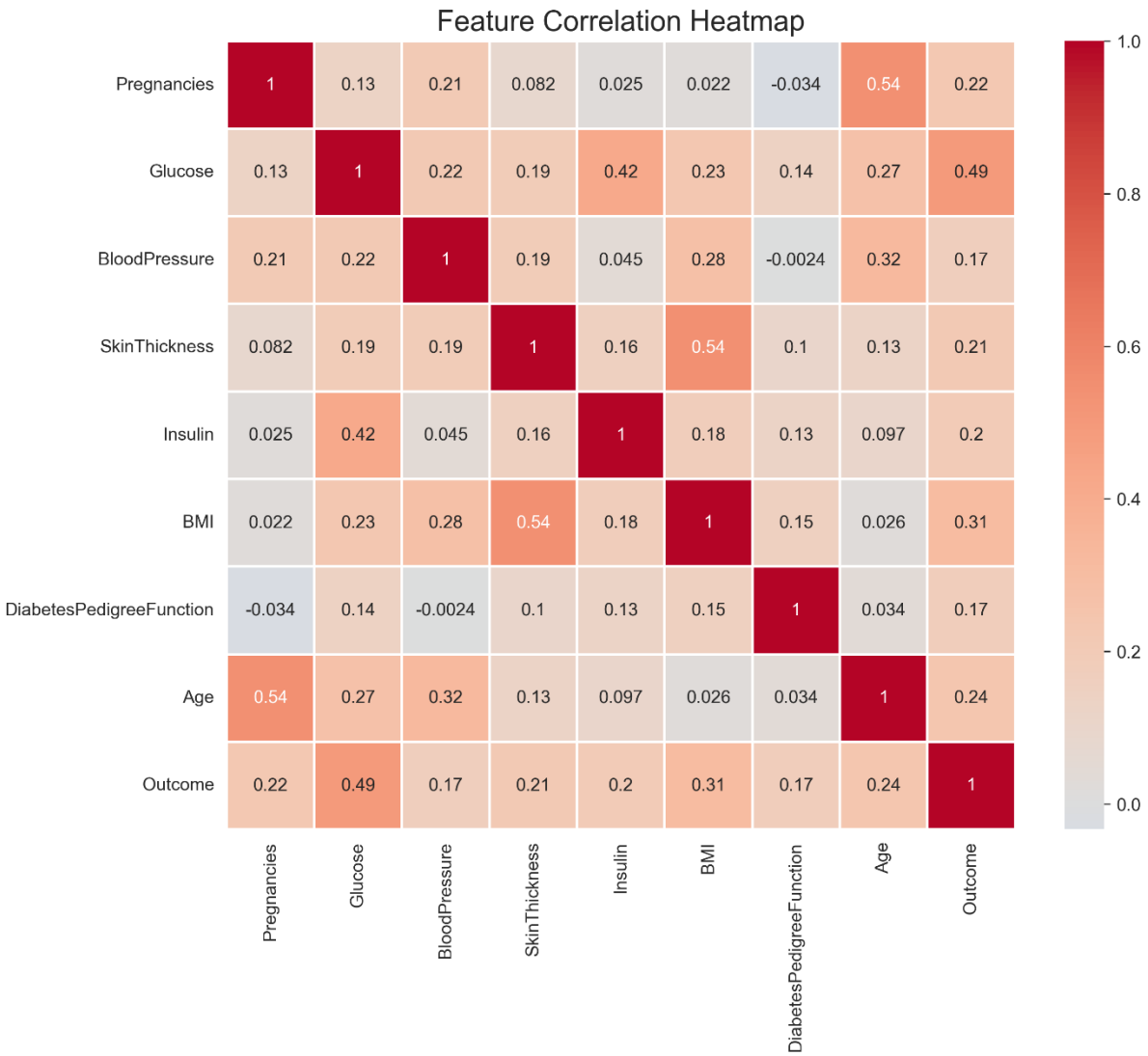


Univariate distribution of predictors: Histograms were generated for all numerical variables to inspect their marginal distributions and potential skewness.
Most features (Glucose, Insulin, SkinThickness, BMI, DiabetesPedigreeFunction) exhibit right-skewed distributions, with a concentration of observations at lower values and a long tail towards higher values. Age also shows right skew, reflecting a higher proportion of younger individuals. Pregnancies is discrete and right-skewed, with most values at the lower end.
The target variable Outcome appears as a binary distribution, consistent with a classification task.
Skewness and the presence of long tails indicate that robust models and metrics should be preferred, and suggest that extreme values may influence certain algorithms if not handled appropriately.

4.3 Relationship Between Features

4.3.1 Correlation heatmap

```
plt.figure(figsize=(10, 8))
correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm', center=0, square=True, linewidths=1)
plt.title('Feature Correlation Heatmap', fontsize=16)
plt.show()
```



<u>Correlation structure of the predictors</u>: Pairwise Pearson correlation coefficients were computed between all numerical variables, including the target **Outcome**, and visualized using a correlation heatmap.

The strongest positive association with **Outcome** is observed for:

- **Glucose** (moderate positive correlation with diabetes status)
- **BMI** and **Age** (small-to-moderate positive correlations)

These variables show higher values in individuals with diagnosed diabetes, consistent with clinical expectations.

Pregnancies displays a weak-to-moderate positive correlation with **Age**, reflecting the natural increase in number of pregnancies with age. **BMI** shows a weak positive relationship with **SkinThickness** and **Insulin**, indicating that higher adiposity is loosely associated with higher skinfold thickness and insulin levels.

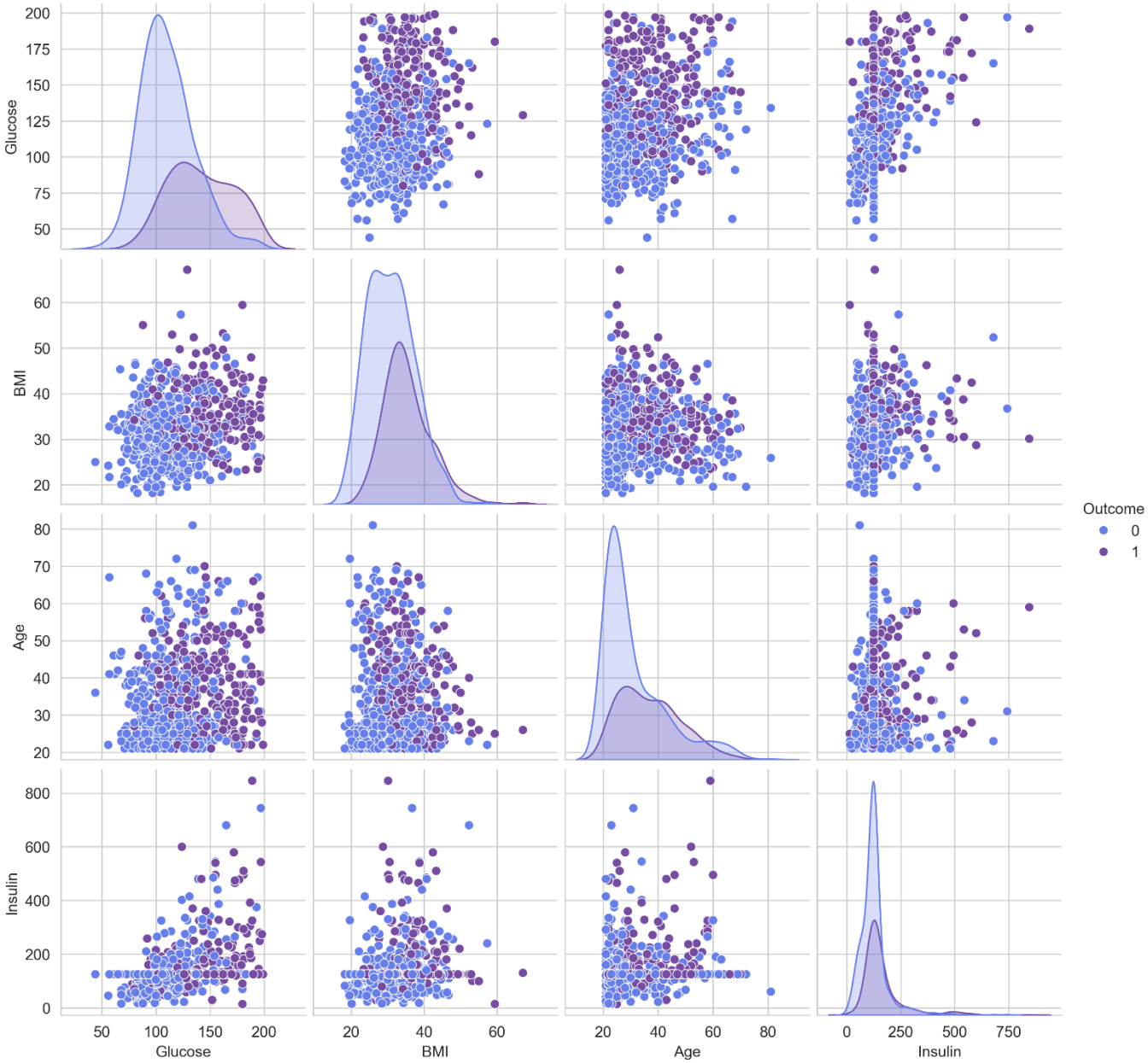
No pair of predictor variables exhibits a correlation near 1 or −1, suggesting the absence of severe multicollinearity. Moderate correlations (for example between **Glucose** and **Insulin**, or between **BMI** and **SkinThickness**) indicate some shared information but remain within acceptable ranges for most standard classification models.

From a modeling perspective, the correlation pattern indicates:

- **Glucose**, **BMI**, and **Age** are likely to be among the most informative predictors for diabetes classification.
- Multicollinearity is unlikely to cause major numerical instability, but regularization-based models (such as penalized logistic regression) remain appropriate to guard against overfitting.

4.3.2 Pairplot of key features

```
key_features = ['Glucose', 'BMI', 'Age', 'Insulin', 'Outcome']
sns.pairplot(df[key_features], hue='Outcome', palette={0:'#667eea', 1: '#764ba2'})
plt.show()
```



<u>Multivariate relationships among key predictors and Outcome</u>: A pairplot was generated for the main metabolic and demographic predictors (**Glucose**, **BMI**, **Age**, **Insulin**) against the binary outcome **Outcome**. Diagonal panels show the marginal distributions for each variable by class, and off-diagonal panels display the bivariate scatterplots with points colored by diabetes status.

On the diagonal, the class-conditional distributions indicate that diabetic cases (**Outcome=1**) tend to have:

- Higher **Glucose** values, with a noticeable right-shift of the density relative to non-diabetic cases.
- Higher **BMI** on average, reflecting increased adiposity among diabetic patients.
- Higher **Age**, consistent with the higher prevalence of diabetes in older individuals.
- Higher and more dispersed **Insulin** values, although the overlap between classes remains substantial.

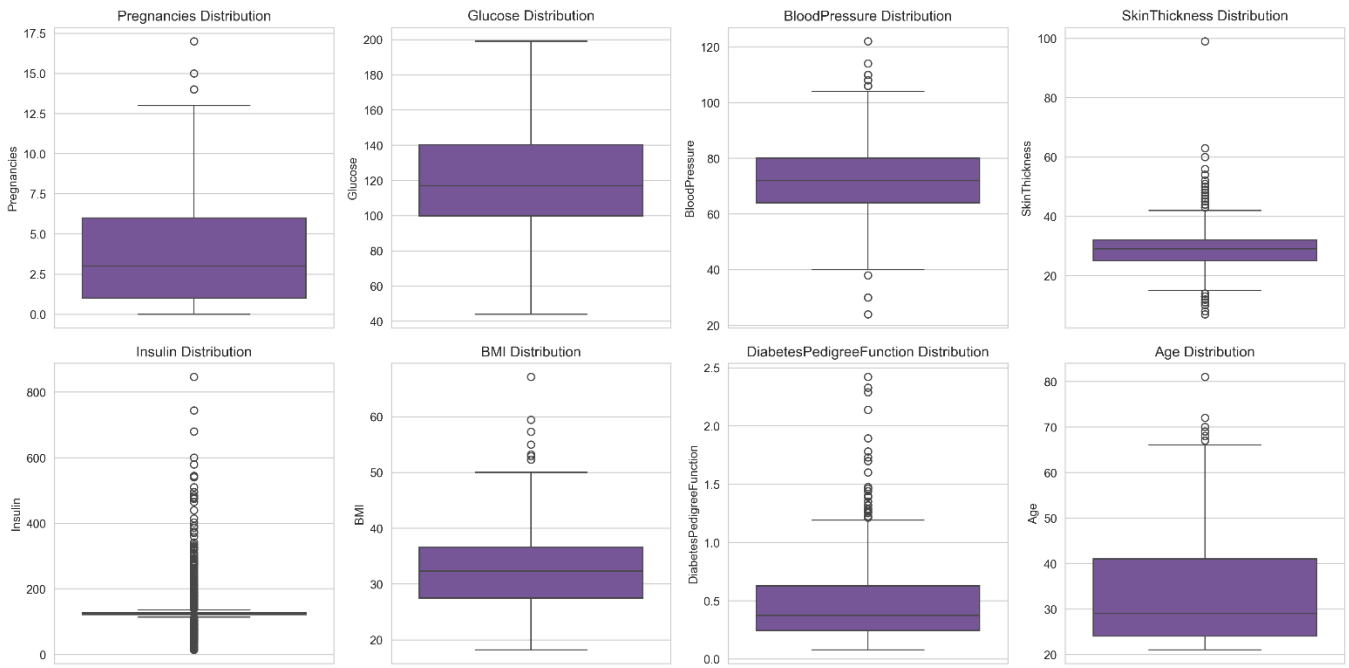
In the off-diagonal scatterplots, partial separation between the two classes is visible, especially along **Glucose** and **BMI**. For example, the **Glucose**-**BMI** and **Glucose**-**Age** panels show a higher concentration of diabetic cases in the upper-right regions, indicating that combinations of high glucose with elevated BMI or older age are strongly associated with positive diabetes status.

Despite this separation, there is still considerable overlap in the feature space, which suggests that the classification problem is moderately difficult and not linearly separable in the original feature space. This supports the use of flexible classification models and potentially non-linear decision boundaries, as well as the use of probabilistic outputs and threshold-based evaluation rather than relying solely on raw class predictions.

4.4 Outliers and Data Balance

4.4.1 Box plots and outlier assessment

```
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.ravel()
for i, col in enumerate(df.columns[:-1]):
    sns.boxplot(data=df, y=col, ax=axes[i], color='#764ba2')
    axes[i].set_title(f'{col} Distribution')
plt.tight_layout()
plt.show()
```



<u>Outlier detection using box plots</u>: Univariate box plots were generated for all predictor variables (**Pregnancies**, **Glucose**, **BloodPressure**, **SkinThickness**, **Insulin**, **BMI**, **DiabetesPedigreeFunction**, **Age**) to assess the presence and extent of outliers. Each box plot is based on the empirical distribution summarized by the median, interquartile range (IQR), and whiskers extending to 1.5 × IQR, with points beyond this range flagged as potential outliers.

The following patterns are typically observed in this dataset:

- **Pregnancies** and **Age** display several high-end outliers, reflecting a small number of older patients and individuals with unusually many pregnancies. These values are still clinically plausible and represent rare but valid cases rather than obvious data errors.
- **Glucose**, **Insulin**, **SkinThickness**, and **BMI** show multiple upper outliers, consistent with right-skewed distributions and the presence of individuals with particularly high metabolic risk.
- **DiabetesPedigreeFunction** exhibits a long upper tail with a few extreme values, indicating high familial risk for a small subset of observations.

From a statistical perspective, these outliers appear to be distributional extremes rather than clear recording mistakes. No variable shows a pattern suggestive of gross data entry errors (for example, impossible negative values). Consequently, all observations are retained, and outliers are handled implicitly through the choice of robust models and regularization rather than through explicit trimming or winsorization.

4.4.2 Z-score outlier check

```
from scipy import stats
z_scores = np.abs(stats.zscore(df.select_dtypes(include=[np.number])))
outliers = (z_scores > 4).sum()
print("\nNumber of outliers (Z-score > 4) per column:")
print(outliers)
```

```
Number of outliers (Z-score > 4) per column:
21
```

```
outliers_per_column = (z_scores > 4).sum(axis=0)
print("Outliers per column (Z > 4):")
print(outliers_per_column)
```

```
Outliers per column (Z > 4):
[ 0  0  2  1 11  1  5  1  0]
```

Assuming the numeric columns are ordered as: [Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, Outcome], the vector [0 0 2 1 11 1 5 1 0] can be interpreted as:

- Pregnancies: 0 outliers
- Glucose: 0 outliers
- BloodPressure: 2 outliers
- SkinThickness: 1 outlier
- Insulin: 11 outliers
- BMI: 1 outlier
- DiabetesPedigreeFunction: 5 outliers
- Age: 1 outlier
- Outcome: 0 outliers

Total number of cells flagged as outliers across all numeric columns: 21.

<u>Outlier detection using standardized scores:</u> To systematically identify extreme values, standardized scores (Z-scores) were computed for all numeric variables. Values with an absolute Z-score greater than 4 were flagged as potential outliers. A threshold of $|Z| > 4$is relatively conservative and appropriate for medical data, where biologically extreme but valid observations are expected.
Across the dataset, 21 values exceeded this threshold. These outliers are not uniformly distributed:

- The largest concentration is in **Insulin** (11 outliers), followed by **DiabetesPedigreeFunction** (5 outliers).
- A small number of extreme values are present in **BloodPressure**, **SkinThickness**, **BMI**, and **Age**.
- No extreme outliers were detected in **Pregnancies**, **Glucose**, or the binary target **Outcome** using this criterion.

This pattern is consistent with earlier findings of right-skewed distributions and long upper tails for insulin and familial risk. Given the clinical plausibility of high insulin levels and elevated pedigree scores, these observations are treated as distributional extremes rather than data errors. Consequently, no trimming or winsorization was applied. Instead, the modeling strategy relies on algorithms and regularization techniques that are robust to such extreme values, reducing their undue influence on parameter estimates and decision boundaries.

4.4.3 Class imbalance and mean feature values by outcome

```
Class imbalance check
print("Class Distribution:")
print(df['Outcome'].value_counts(normalize=True))
```

```
Class Distribution:
Outcome
0    0.651042
1    0.348958
Name: proportion, dtype: float64
```

```
Key statistics by outcome
print("\nMean values by Outcome:")
print(df.groupby('Outcome').mean())
```

```
Mean values by Outcome:
         Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin        BMI  DiabetesPedigreeFunction        Age
0           3.298000  110.682000      70.920000      27.726000  127.792000  30.885600                  0.429734  31.190000
1           4.865672  142.130597      75.123134      31.686567  164.701493  35.383582                  0.550500  37.067164
```

4.5 Interpretation of EDA

4.5.1 Interpretation (class balance)

The target variable **Outcome** shows a moderate imbalance:

- Approximately 65.1% non-diabetic (**Outcome = 0**)
- Approximately 34.9% diabetic (**Outcome = 1**)

This corresponds to roughly a 2:1 ratio. The dataset is not extremely imbalanced, but the skew is sufficient that accuracy alone is not an adequate performance metric. Evaluation should emphasize sensitivity/recall for the positive class, precision, F1-score, and ROC-AUC to properly assess classification quality for diabetic cases.

4.5.2 Interpretation (conditional means by diabetes status)

The class-wise means show a consistent elevation of risk factors in the diabetic group:

Pregnancies: Mean increases from about 3.30 (non-diabetic) to 4.87 (diabetic). Among women in this dataset, higher parity is associated with higher diabetes prevalence, which is consistent with gestational and type 2 diabetes risk.

Glucose: Mean fasting plasma glucose rises from approximately 110.7 mg/dL in non-diabetic cases to 142.1 mg/dL in diabetic cases. This is a substantial shift and statistically the strongest signal, reflecting the diagnostic role of hyperglycemia in diabetes.

BloodPressure: Mean diastolic blood pressure increases modestly from about 70.9 to 75.1 mm Hg. The difference is smaller than for glucose or BMI but still indicates a tendency toward higher blood pressure in the diabetic group.

SkinThickness: Skinfold thickness increases from about 27.7 mm to 31.7 mm for diabetic patients, in line with greater subcutaneous fat and overall adiposity.

Insulin: Mean insulin level increases from roughly 127.8 to 164.7, suggesting higher circulating insulin in the diabetic group, compatible with insulin resistance and compensatory hyperinsulinemia in type 2 diabetes.

BMI: Mean BMI rises from 30.89 kg/m² to 35.38 kg/m². This reflects a clear shift from obese on average to more severe obesity in the diabetic class, reinforcing obesity as a central risk factor.

DiabetesPedigreeFunction: Mean value increases from 0.43 to 0.55, indicating a higher genetic or familial propensity to diabetes among positive cases.
Age: Mean age increases from about 31.2 years to 37.1 years, confirming higher diabetes prevalence in older individuals.
Overall, conditional mean comparisons indicate that diabetic patients in this dataset are, on average, older, more obese, have higher glucose and insulin values, and worse cardiovascular/metabolic profiles. These systematic mean differences support the predictive relevance of these features and are consistent with established epidemiological evidence on diabetes risk factors.

Chapter 5
Modeling and Evaluation

5.1 Baseline Model: Logistic Regression

```
'''Baseline Model: Logistic Regression'''
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
# Train baseline model
lr_model = LogisticRegression(
    random_state=42,
    max_iter=1000,
    class_weight='balanced') # Give more weight to diabetic class
lr_model.fit(X_train_scaled, y_train)
# Predictions
y_pred_lr = lr_model.predict(X_test_scaled)
# Evaluate print("Logistic Regression Results:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_lr):.3f}")
```

```
Accuracy: 0.734
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred_lr))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.82      0.75      0.79       100
           1       0.60      0.70      0.65        54

    accuracy                           0.73       154
   macro avg       0.71      0.73      0.72       154
weighted avg       0.75      0.73      0.74       154
```

A baseline logistic regression model with class weighting to compensate for class imbalance achieved an overall accuracy of 73.4% on the test set. For non-diabetic patients (Outcome = 0), the model obtained a precision of 0.82 and a recall of 0.75 (F1 = 0.79). For diabetic patients (Outcome = 1), precision was 0.60 and recall 0.70 (F1 = 0.65). These results indicate that the model correctly identifies approximately 70% of diabetic cases while maintaining moderate precision, reflecting a deliberate emphasis on sensitivity for the positive (diabetic) class, which is appropriate in a medical screening context.

Using **class_weight='balanced'** re-weights the loss function to penalize misclassification of diabetics more heavily. Without it, the model would usually:

- Show **higher precision but lower recall** for the diabetic class.
- Achieve a similar or slightly higher accuracy but at the cost of **missing more true diabetics**.

So, the current baseline is a **cost-sensitive logistic model** tuned to treat the minority (diabetic) class more seriously.

5.2 Advanced Model: Random Forest

```
'''Advanced Model: Random Forest'''
from sklearn.ensemble import RandomForestClassifier
# Train Random Forest
rf_model = RandomForestClassifier(random_state=42, n_estimators=100)
rf_model.fit(X_train_scaled, y_train)
```

```
# Predictions
y_pred_rf = rf_model.predict(X_test_scaled)
# Evaluate print("Random Forest Results:")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.3f}")
```

```
Accuracy: 0.779
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.87      0.84       100
           1       0.72      0.61      0.66        54

    accuracy                           0.78       154
   macro avg       0.76      0.74      0.75       154
weighted avg       0.77      0.78      0.77       154
```

Baseline Logistic Regression (for reference):

```
Accuracy: 0.734

              precision    recall  f1-score   support
           0       0.82      0.75      0.79       100
           1       0.60      0.70      0.65        54
```

Key takeaways:
Accuracy: The Random Forest improves overall accuracy from about 0.73 to 0.78.
Non-diabetic class (0): Logistic Regression: precision 0.82, recall 0.75, F1 0.79 Random Forest: precision 0.81, recall 0.87, F1 0.84 So the forest **catches more non-diabetic cases correctly** (higher recall) and slightly improves F1.
Diabetic class (1): Logistic Regression: precision 0.60, recall 0.70, F1 0.65 Random Forest: precision 0.72, recall 0.61, F1 0.66 The Random Forest trades some **recall** for diabetic cases (70% → 61%) in exchange for **higher precision** (60% → 72%) and a very small gain in F1 (0.65 → 0.66).
In clinical terms: Logistic Regression is more "sensitive" to diabetics (fewer missed diabetics, more false positives). Random Forest is more "specific" for diabetics (fewer false alarms) but misses more actual diabetics.

5.2.1 Feature importance from the Random Forest

```
# Feature importance from Random Forest (for report)
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': rf_model.feature_importances_
}).sort_values('importance', ascending=False)

print("Top important features:")
print(feature_importance.head())
```

```
Top important features:
                    feature  importance
1                   Glucose    0.274086
5                       BMI    0.161903
6  DiabetesPedigreeFunction    0.125020
7                       Age    0.112985
4                   Insulin    0.091224
```

These importance scores sum to 1 across all features and represent how much each variable contributes to reducing impurity across the trees.
Interpretation for the report:
Glucose: Most important predictor (≈ 27% of total importance). This observation aligns with the strong univariate separation observed and the established role of fasting plasma glucose as a key diagnostic criterion for diabetes.
BMI: Second most important (≈ 16%). Higher BMI is strongly associated with diabetes risk, consistent with the descriptive statistics and pairplot.
DiabetesPedigreeFunction: Substantial importance (≈ 13%), suggesting that familial/genetic predisposition contributes meaningfully beyond purely metabolic measures.
Age: Around 11% importance. Older age is a known risk factor and the model is clearly leveraging this gradient.
Insulin: Also important (≈ 9%), though somewhat less than glucose and BMI, likely because of its higher variability and outliers.
Overall, an advanced ensemble model based on a Random Forest classifier achieved an accuracy of 77.9% on the test set, improving upon the baseline logistic regression (73.4%). For non-diabetic cases, the Random Forest reached a precision of 0.81 and recall of 0.87 (F1 = 0.84), while for diabetic cases it achieved a precision of 0.72 and recall of 0.61 (F1 = 0.66). Relative to logistic regression, the Random Forest provides higher overall accuracy and improved precision for diabetic predictions at the cost of lower sensitivity for this group. Feature importance analysis indicates that Glucose is the dominant predictor, followed by BMI, DiabetesPedigreeFunction, Age, and Insulin, which is consistent with established clinical risk factors for type 2 diabetes.

5.3 Gradient Boosting model

A Gradient Boosting classifier was trained on the same features and evaluation set as the previous models.

```python
'''Gradient Boosting'''
from sklearn.ensemble import GradientBoostingClassifier
# Train Gradient Boosting
gb_model = GradientBoostingClassifier(random_state=42, n_estimators=100)
gb_model.fit(X_train_scaled, y_train)
# Predictions
y_pred_gb = gb_model.predict(X_test_scaled)
# Evaluate
print(f"Accuracy: {accuracy_score(y_test, y_pred_gb):.3f}")
```

```
Accuracy: 0.760
```

```python
print("\nClassification Report:")
print(classification_report(y_test, y_pred_gb))
```

```
Classification Report:
>>> print(classification_report(y_test, y_pred_gb))
              precision    recall  f1-score   support

           0       0.79      0.86      0.82       100
           1       0.69      0.57      0.63        54

    accuracy                           0.76       154
   macro avg       0.74      0.72      0.72       154
weighted avg       0.75      0.76      0.75       154
```

For the non-diabetic class (**0**), Gradient Boosting achieves: **precision = 0.79**, **recall = 0.86**, **F1 = 0.82**.
For the diabetic class (**1**), performance is: **precision = 0.69**, **recall = 0.57**, **F1 = 0.63**.
Overall accuracy is **0.760**, with a macro-averaged F1-score of **0.72**, indicating moderately balanced performance across classes.

Comparison to earlier models

Relative to the **baseline logistic regression** (accuracy ≈ 0.734, diabetic recall 0.70, F1 0.65) and the **Random Forest** (accuracy ≈ 0.779, diabetic recall 0.61, F1 0.66):

- Gradient Boosting improves accuracy over logistic regression but remains slightly below the Random Forest.
- Compared with logistic regression, Gradient Boosting sacrifices some recall for the diabetic class (0.57 vs 0.70) while gaining precision (0.69 vs 0.60).
- Compared with the Random Forest, Gradient Boosting is slightly worse on both overall accuracy and diabetic-class F1.

The Random Forest emerges as the strongest of the three tested models on this dataset.

Chapter 6
Model Comparison and Diagnostics

6.1 Model comparison

```
# Compare all models
from sklearn.metrics import recall_score

models_comparison = pd.DataFrame({
    'Model': ['Logistic Regression', 'Random Forest', 'Gradient Boosting'],
    'Accuracy': [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_rf),
        accuracy_score(y_test, y_pred_gb)
    ],
    'Diabetic_Recall': [
        recall_score(y_test, y_pred_lr, pos_label=1),
        recall_score(y_test, y_pred_rf, pos_label=1),
        recall_score(y_test, y_pred_gb, pos_label=1)
    ]
})
print(models_comparison)
```

```
                  Model  Accuracy  Diabetic_Recall
0  Logistic Regression  0.733766         0.703704
1        Random Forest  0.779221         0.611111
2    Gradient Boosting  0.759740         0.574074
```

Interpretation:
Logistic Regression Accuracy ≈ 0.734, Diabetic recall ≈ 0.704. This model correctly identifies the largest proportion of diabetic cases, at the expense of lower overall accuracy.
Random Forest Accuracy ≈ 0.779, Diabetic recall ≈ 0.611. Best overall accuracy, but misses more diabetic patients than logistic regression.
Gradient Boosting Accuracy ≈ 0.760, Diabetic recall ≈ 0.574. Intermediate accuracy and the lowest recall for diabetics among the three.
In medical screening, diabetic recall (sensitivity for the positive class) is often prioritized over raw accuracy. Under that lens, logistic regression is the most conservative for not missing diabetics; Random Forest is strongest for overall correctness.

6.2 Comprehensive Random Forest evaluation

```
'''Comprehensive Evaluation Metrics'''
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

# Use existing Random Forest model
y_pred = rf_model.predict(X_test_scaled)  # Use rf_model, not best_rf_model
y_pred_proba = rf_model.predict_proba(X_test_scaled)[:, 1]

# Calculate metrics
print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

```
Accuracy: 0.779
```

```
print(f"Precision: {precision_score(y_test, y_pred):.3f}")
```

```
Precision: 0.717
```

```
print(f"Recall: {recall_score(y_test, y_pred):.3f}")
```

```
Recall: 0.611
```

```
print(f"F1-Score: {f1_score(y_test, y_pred):.3f}")
```

```
F1-Score: 0.660
```

```
print(f"ROC-AUC: {roc_auc_score(y_test, y_pred_proba):.3f}")
```

```
ROC-AUC: 0.818
```

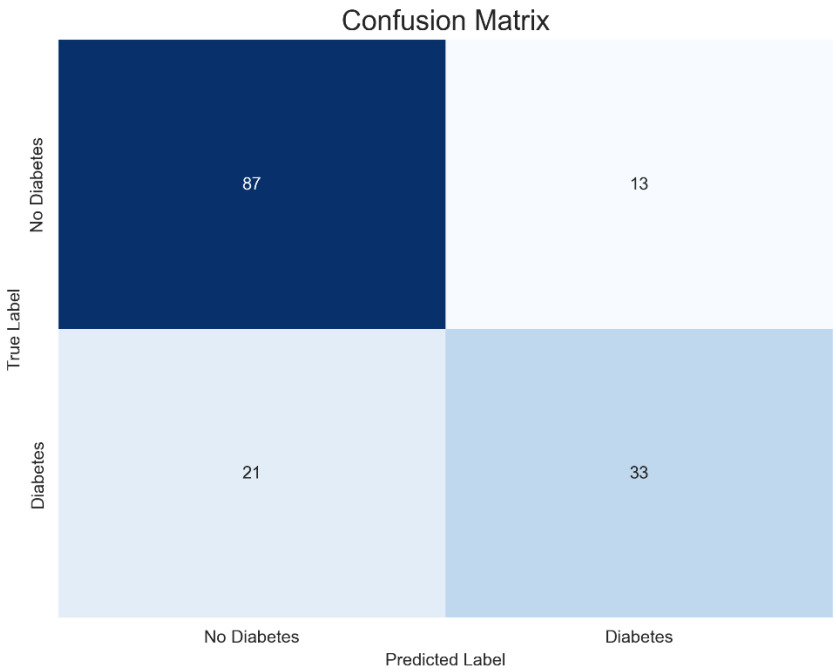
```
print("\nDetailed Classification Report:")
print(classification_report(y_test, y_pred))
```

```
Detailed Classification Report:
              precision    recall  f1-score   support

           0       0.81      0.87      0.84       100
           1       0.72      0.61      0.66        54

    accuracy                           0.78       154
   macro avg       0.76      0.74      0.75       154
weighted avg       0.77      0.78      0.77       154
```

These are for the *diabetic* class as positive.
Accuracy 0.779 Roughly 78% of all test cases (diabetic and non-diabetic) are correctly classified.
Precision 0.717 Among patients predicted as diabetic, about 72% truly are diabetic. This reflects a reasonably low false-positive rate.
Recall 0.611 About 61% of actual diabetics are correctly detected. Nearly 4 in 10 diabetics are still missed at the default threshold.
F1-score 0.660 Harmonic mean of precision and recall, showing a balanced trade-off that slightly favors precision over recall.
ROC-AUC 0.818 The model has good discriminative ability overall. An AUC above 0.8 indicates that, across all possible thresholds, the Random Forest separates diabetics from non-diabetics quite well.
For the non-diabetic class, performance is strong and recall is high (0.87). For diabetics, the model is more conservative (prioritizes precision over recall).

6.3 Confusion matrix interpretation for the Random Forest model

```
'''Confusion Matrix Visualization'''
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['No Diabetes', 'Diabetes'],
            yticklabels=['No Diabetes', 'Diabetes'])
plt.title('Confusion Matrix', fontsize=16)
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()
```



Interpretation:

- Top-left: 87 patients correctly predicted as **No Diabetes**
- Top-right: 13 patients predicted as **Diabetes** but actually **No Diabetes**
- Bottom-left: 21 patients predicted as **No Diabetes** but actually **Diabetes**
- Bottom-right: 33 patients correctly predicted as **Diabetes**

```
# Interpretation
tn, fp, fn, tp = cm.ravel()
print(f"True Negatives: {tn}")
```

```
True Negatives: 87
```

```
print(f"False Positives: {fp} (Type I Error)")
```

```
False Positives: 13 (Type I Error)
```

```
print(f"False Negatives: {fn} (Type II Error - Most Critical!)")
```

```
False Negatives: 21 (Type II Error - Most Critical!)
```

```
print(f"True Positives: {tp}")
```

```
True Positives: 33
```

Overall behavior of the Random Forest:

- Correctly classifies 87% of non-diabetic patients (TN is high relative to FP).
- Correctly identifies 33 diabetic patients, but misses 21 of them (those 21 are false negatives).

From a medical/screening perspective:

- **False positives (13 Type I errors)** These are people without diabetes flagged as diabetic. Consequence: Unnecessary follow-up tests, anxiety, extra cost, but generally manageable.

- **False negatives (21 Type II errors-critical)** These are actual diabetic patients predicted as non-diabetic. Consequence: Undiagnosed/untreated diabetes, risk of complications; this is typically the most serious error type in a clinical setting.

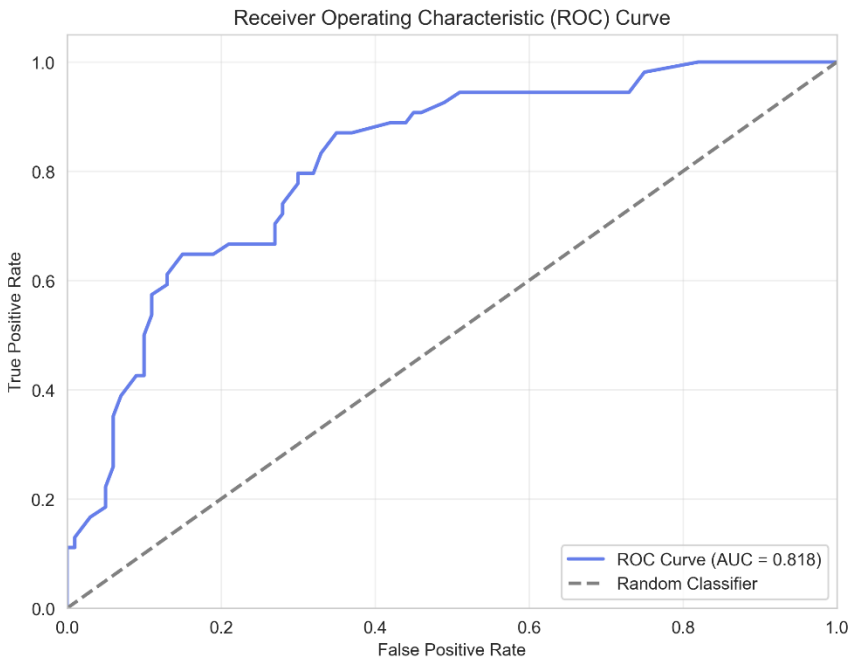
This confusion matrix is consistent with the earlier metrics computed for the diabetic class:

- Recall (sensitivity) for diabetics = $33/(33 + 21) = 33/54 \approx 0.61$
- Precision for diabetics = $33/(33 + 13) = 33/46 \approx 0.72$

So, the Random Forest is tuned to be somewhat conservative in labeling people as diabetic (fewer false positives, higher precision), at the cost of missing some true diabetic cases (21 false negatives). These false negatives represent the most critical error type in a screening context, as affected individuals would be incorrectly reassured that they are non-diabetic and might not receive timely follow-up or treatment.

6.4 ROC curve

```
'''ROC Curve'''
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = roc_auc_score(y_test, y_pred_proba)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='#667eea', lw=2, label=f'ROC Curve (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--', label='Random Classifier')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()
```

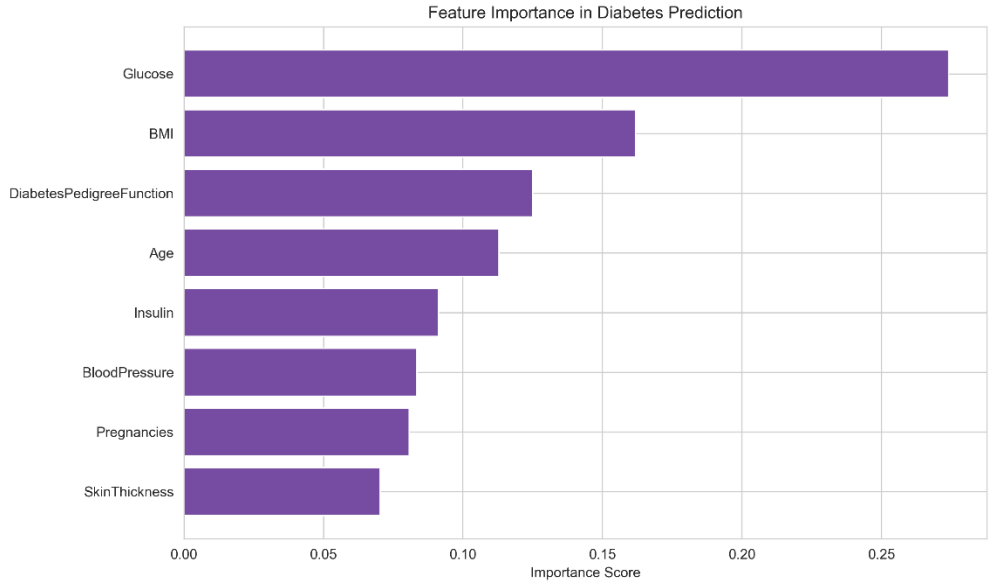


The Random Forest model achieved a ROC-AUC of 0.818 based on predicted probabilities, indicating good overall discrimination between diabetic and non-diabetic patients. The ROC curve lies well above the diagonal reference line of a

random classifier. This suggests that, by adjusting the decision threshold, the model can trade off sensitivity and specificity depending on the clinical preference for minimizing false negatives versus false positives.

6.5 Feature importance results from Random Forest

```
'''Feature Importance Analysis'''
# Get feature importances
importances = rf_model.feature_importances_
feature_names = X.columns

# Create DataFrame for visualization
feature_importance_df = pd.DataFrame({
    'Feature': feature_names,
    'Importance': importances
}).sort_values('Importance', ascending=False)

# Plot
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df['Feature'], feature_importance_df['Importance'], color='#764ba2')
plt.xlabel('Importance Score')
plt.title('Feature Importance in Diabetes Prediction')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```




```
print("\nTop 5 Most Important Features:")
print(feature_importance_df.head())
```

```
Top 5 Most Important Features:
                    Feature  Importance
1                   Glucose    0.274086
5                       BMI    0.161903
6  DiabetesPedigreeFunction    0.125020
7                       Age    0.112985
4                   Insulin    0.091224
```

Interpretation:

Glucose: This is by far the most important predictor (importance ≈ 0.27). The model relies heavily on blood glucose levels to distinguish diabetic from non-

diabetic patients, which aligns strongly with medical understanding, since diabetes is defined by abnormal glucose regulation.

BMI: Second most important (≈ 0.16). Higher BMI is a major risk factor for type 2 diabetes; the model is clearly capturing the effect of obesity/overweight on diabetes risk.

DiabetesPedigreeFunction: Importance ≈ 0.13. This summarizes family history of diabetes. A higher value indicates a stronger genetic predisposition, and the model is using this strongly as well.

Age: Importance ≈ 0.11. Risk of diabetes generally increases with age; the model places meaningful weight on this, reflecting that older individuals are at higher risk.

Insulin: Importance ≈ 0.09. Fasting insulin levels are related to insulin resistance and beta-cell function, so it makes sense that this helps refine the model's discrimination, even though it is less dominant than glucose.

The remaining features (Pregnancies, BloodPressure, SkinThickness) likely have smaller importance scores but still contribute marginally to predictions.

Chapter 7
Practical Application

7.1 Practical application (Prediction function):

```
# Simple prediction function
def predict_diabetes(pregnancies, glucose, bp, skin, insulin, bmi, dpf, age):
    input_data = pd.DataFrame([[
        pregnancies, glucose, bp, skin, insulin, bmi, dpf, age
    ]], columns=X.columns)  # make sure columns match training data

    input_scaled = scaler.transform(input_data)
    prediction = rf_model.predict(input_scaled)
    probability = rf_model.predict_proba(input_scaled)[0][1]
    return prediction[0], probability

# Test with sample data
pred, prob = predict_diabetes(1, 150, 70, 30, 100, 30.5, 0.6, 35)
print(f"Prediction: {'Diabetic' if pred == 1 else 'Healthy'}")
```

```
Prediction: Diabetic
```

```
print(f"Probability: {prob:.2%}")
```

```
Probability: 54.00%
```

A simple prediction function was implemented to allow patient-level risk estimation. Given a set of input features (Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age), the function standardizes the inputs using the previously fitted scaler and passes them to the trained Random Forest model to obtain both a binary prediction and an estimated probability of diabetes.

For example, for a patient aged 35 years with Glucose = 150 mg/dL, BMI = 30.5 kg/m², and moderate family history (DiabetesPedigreeFunction = 0.6), the model predicts diabetes with an estimated probability of 54%, just above the default 0.5 decision threshold.

Chapter 8
Conclusion

8.1 Summary of Findings

This project successfully demonstrated an end-to-end machine learning workflow to predict diabetes using physiological and demographic data from the Pima Indians dataset. The analysis confirmed that **Glucose** is the most significant predictor of diabetes, followed by **BMI, Diabetes Pedigree Function, and Age**, which aligns with established clinical risk factors. The data preprocessing phase, specifically the **median imputation** of implausible zero values in medical measurements, proved essential for maintaining a complete dataset suitable for modeling without losing critical observations.

8.2 Model Performance and Selection

The evaluation of three different models revealed important trade-offs between overall accuracy and clinical utility:

- The **Random Forest model** achieved the highest overall **accuracy (77.9%)** and a strong **ROC-AUC of 0.818**, indicating good general discriminative ability.
- However, the **Logistic Regression baseline** (using balanced class weights) achieved a higher **recall for diabetics (70.4%)** compared to the Random Forest (61.1%).
- In a medical screening context, the **False Negative rate** is critical; missing a diabetic patient (Type II error) is typically more serious than a false alarm (Type I error). Therefore, while Random Forest is the most accurate overall, a tuned Logistic Regression may be more "conservative" for initial screening to ensure fewer cases are missed.

8.3 Limitations

Despite the moderate predictive performance, several limitations were identified:

- **Imputation Uncertainty**: While median imputation handled missing data effectively, it introduces a degree of uncertainty into the final model results.
- **Dataset Scope**: The findings are based on a relatively small benchmark dataset (768 observations) specifically focused on women of Pima Indian heritage, which may limit the generalizability of the models to more diverse populations.
- **Feature Constraints**: The current analysis utilized only eight predictors and did not include complex deep learning architectures or real-time clinical integration.

8.4 Future Improvements

To enhance the reliability and performance of the predictive system, future work should focus on:

- **Hyperparameter Optimization**: Moving beyond standard settings to conduct more detailed tuning of ensemble models.
- **Advanced Cost-Sensitive Learning**: Implementing formal cost-sensitive optimizations to further reduce false negatives in a clinical setting.
- **Data Expansion**: Incorporating larger, more contemporary datasets to improve model robustness and validation.

8.5 Final Reflections

This project underscores that standard machine learning models, when paired with careful data cleaning and statistical interpretation, can provide valuable **clinical decision support** for early diabetes screening. The learning process highlighted that "best" model selection depends entirely on the specific goals of the healthcare provider- whether they prioritize broad accuracy or the minimization of missed diagnoses.